

第3章 STM32F4：高性能的数字信号控制器

3.1 STM32系列以及Cortex-M4内核

3.2 STM32F407芯片系统结构

3.3 最小系统

3.4 GPIO工作原理和寄存器

32位MCU - ARM® Cortex® 内核



Cortex-M4内核



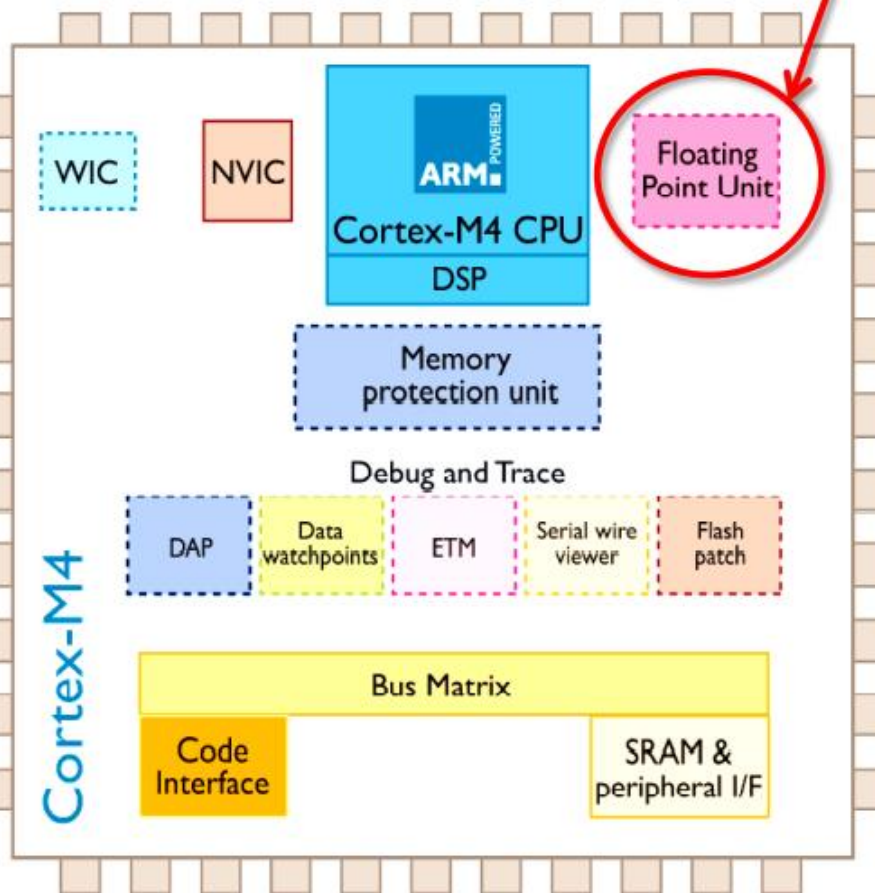
	Cortex-M4	Cortex-M3	Cortex-M0
核心版本	v7ME	v7M	v6M
指令系统	Thumb® / Thumb-2		Thumb® / Thumb-2 subset
指令增强	单周期的 16、32位MAC 单周期的 双16位MAC 8、16位SIMD计算 硬件除法(2~12周期)	硬件除法(2~12周期) 单周期 (32x32)乘法 支持饱和算术运算	可选的硬件单周期 (32x32)乘法
流水线	三级 + 分支推测		三级
执行效率	2.19 CoreMark/MHz, 1.25 DMIPS/MHz		1.62 CoreMark/MHz 0.9 DMIPS/MHz
存储器保护	可选。8区域管理，可划分子区域和后台区		没有
中断	非屏蔽中断(NMI) + 1~240个物理中断源		非屏蔽中断(NMI) + 1~32个物理中断源
中断优先级	8~256个优先级		4级优先
唤醒中断控制器	多达 240个 唤醒中断		可选
睡眠模式	集成 WFI 和 WFE 指令。退出时睡眠功能，睡眠和深度睡眠信号。 使用 ARM 的电源管理部件，可选择保持模式。		
位操作	集成位指令和位带域		
调试	可选的 JTAG 和 SWD 调试接口 支持最多 8个 断点和 4个 察看点		可选的 JTAG 和 SW 调试接口 支持最多 4个 断点和 2个 察看点
跟踪(可选)	指令跟踪(ETM)、数据跟踪(DWT)和仪器跟踪(ITM)模块		

增加单精度浮点运算单元

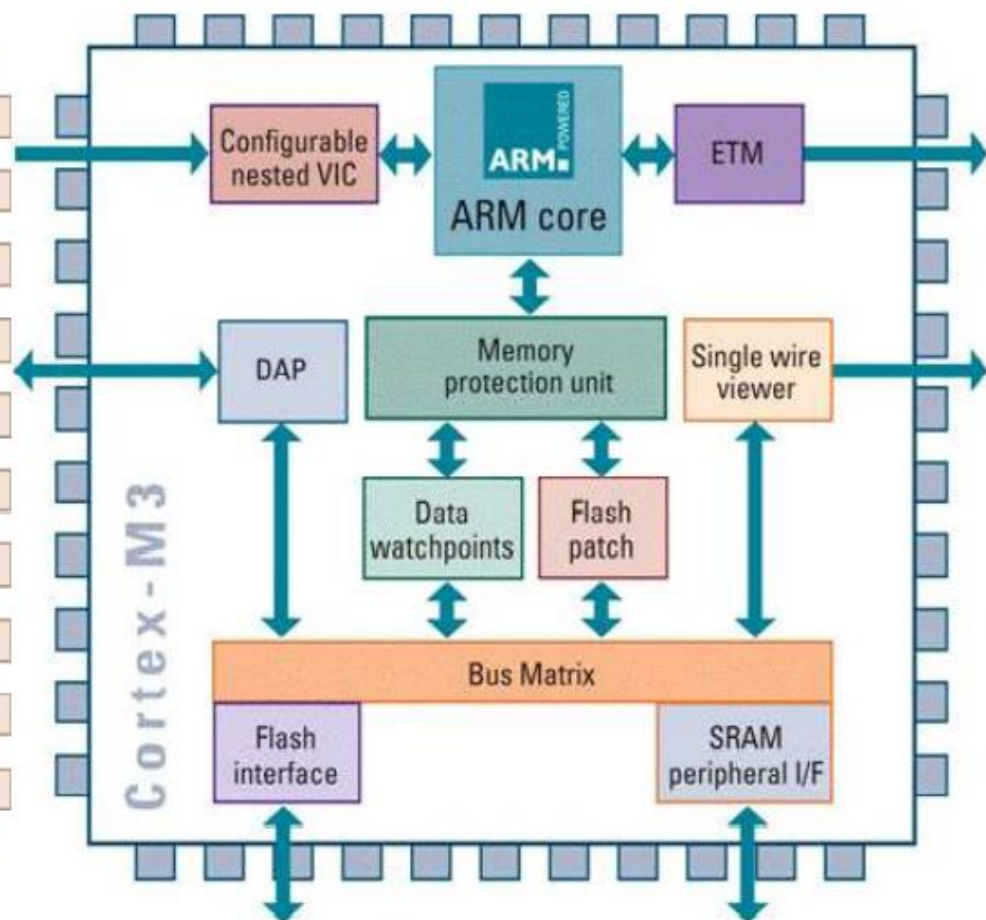


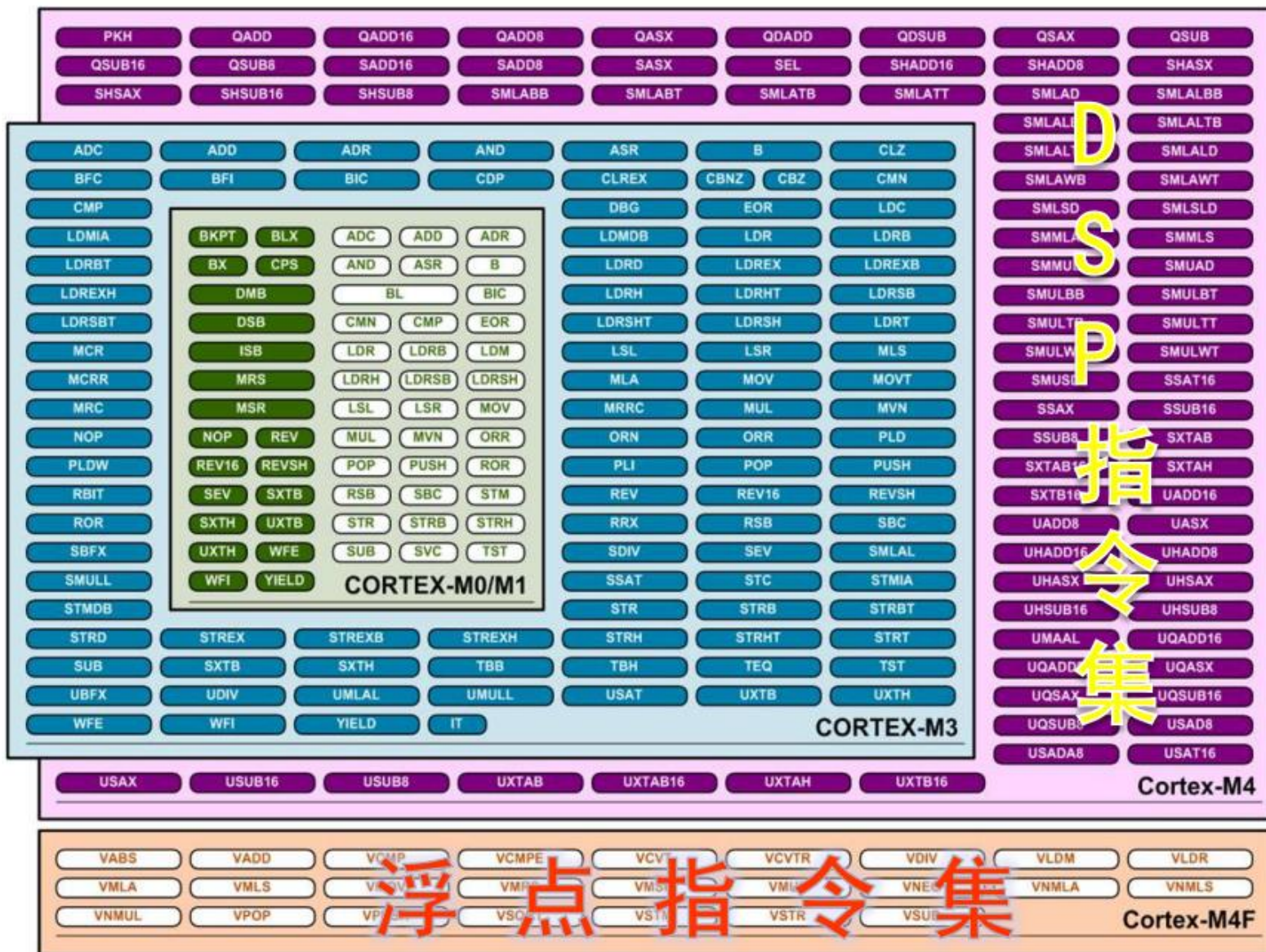
单精度浮点运算单元兼容IEEE 754标准

Cortex-M4架构



Cortex-M3架构





STM32F407/417系列专为医疗、工业和消费类应用而设计

STM32F407/417提供Cortex-M4™内核（带浮点单元）的性能，运行频率为168 MHz。

性能：在168 MHz频率下，STM32F407/417通过闪存提供210 DMIPS/566 CoreMark性能，使用意法半导体的ART加速器实现0等待状态。DSP指令和浮点单元扩大了可寻址应用的范围。

功效：意法半导体的90 nm工艺、ART加速器和动态功耗调节功能使运行模式和从闪存执行的电流消耗在168 MHz时低至238 μ A/MHz。

丰富的连接性：卓越和创新的外设：与STM32F4x5系列相比，STM32F407/417产品线具有以太网MAC10/100和IEEE 1588 v2支持的8至14位并行相机接口，可连接CMOS相机传感器。

- 2 个 USB OTG（一个支持 HS）
- 音频：专用音频锁相环和 2 个全双工 I²S
- 多达 15 个通信接口（包括 6 个 USART 以高达 11.25 Mbit/s 的速度运行，3 个 SPI 以高达 45 Mbit/s 的速度运行，3 个 I²C、2 个 CAN、SDIO）
- 模拟：两个12位DAC，三个12位ADC，在交错模式下达到2.4 MSPS或7.2 MSPS
- 多达 17 个定时器：16 位和 32 位，工作频率高达 168 MHz
- 使用灵活的静态内存控制器轻松扩展内存范围，支持紧凑型闪存、SRAM、PSRAM、NOR 和 NAND 内存
- 模拟真随机数发生器
- STM32F417还集成了一个加密/哈希处理器，为AES 128、192、256、Triple DES和哈希（MD5、SHA-1）提供硬件加速。

集成：STM32F417x产品组合提供从512 KB（仅WLCSP90封装）到1 MB闪存、192 KB SRAM和64至144引脚的封装，小至4 x 4.2 mm。

STM32F407系统架构

主系统由 32 位多层 AHB 总线矩阵构成，可实现以下部分的互连：

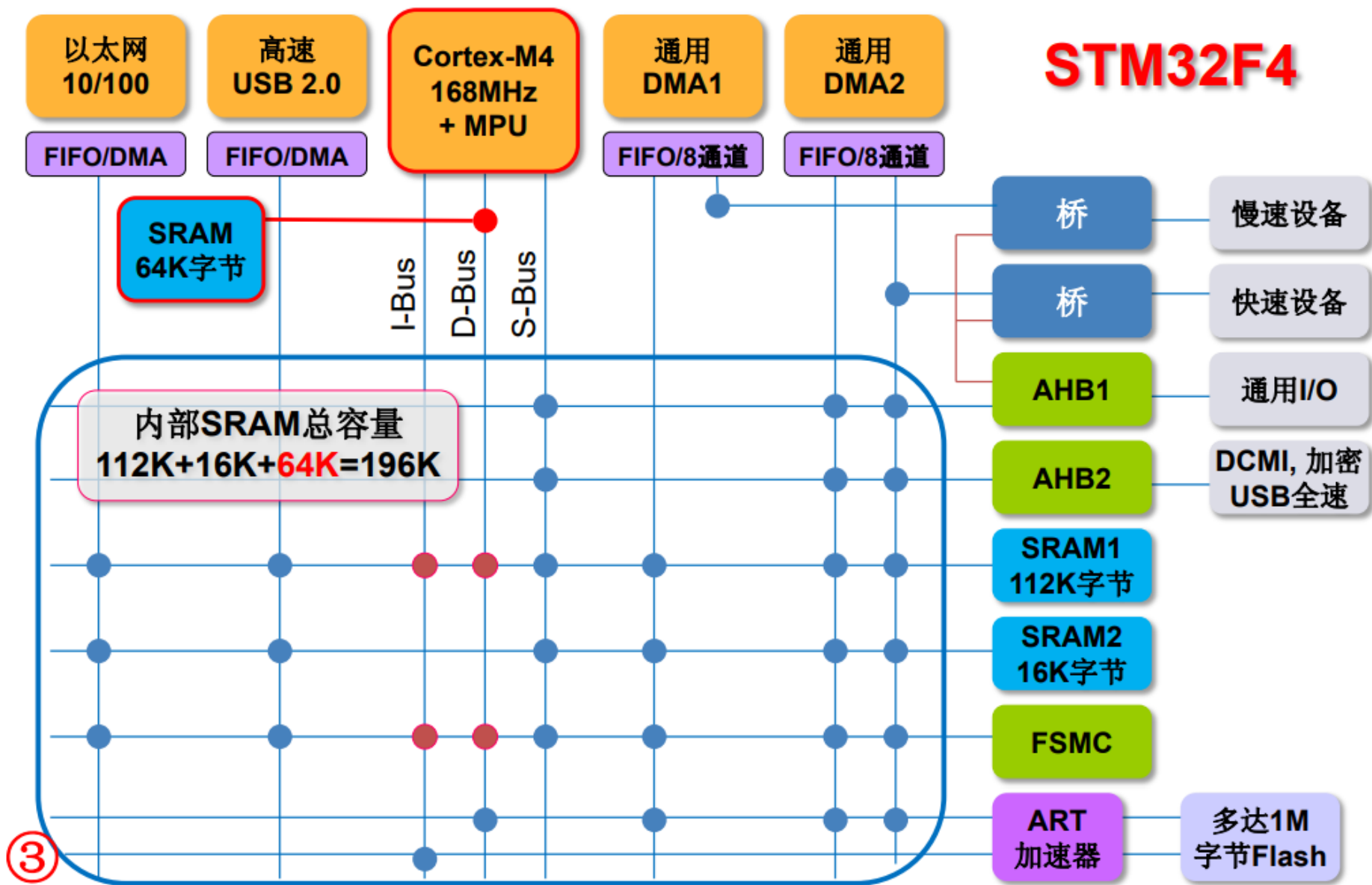
- 八条主控总线：
 - Cortex™-M4F 内核 I 总线、D 总线和 S 总线
 - DMA1 存储器总线
 - DMA2 存储器总线
 - DMA2 外设总线
 - 以太网 DMA 总线
 - USB OTG HS DMA 总线
- 七条被控总线：
 - 内部 Flash ICode 总线
 - 内部 Flash DCode 总线
 - 主要内部 SRAM1 (112 KB)
 - 辅助内部 SRAM2 (16 KB)
 - 辅助内部 SRAM3 (64 KB) (仅适用于 STM32F42xxx 和 STM32F43xxx 器件)
 - AHB1 外设 (包括 AHB-APB 总线桥和 APB 外设)
 - AHB2 外设
 - FSMC

借助总线矩阵，可以实现主控总线到被控总线的访问，这样即使在多个高速外设同时运行期间，系统也可以实现并发访问和高效运行。

32位多重AHB总线矩阵



STM32F4



STM32F407系统架构

➤ I 总线

此总线用于将 Cortex™-M4F 内核的指令总线连接到总线矩阵。内核通过此总线获取指令。此总线访问的对象是包含代码的存储器（内部 Flash/SRAM 或通过 FSMC 的外部存储器）。

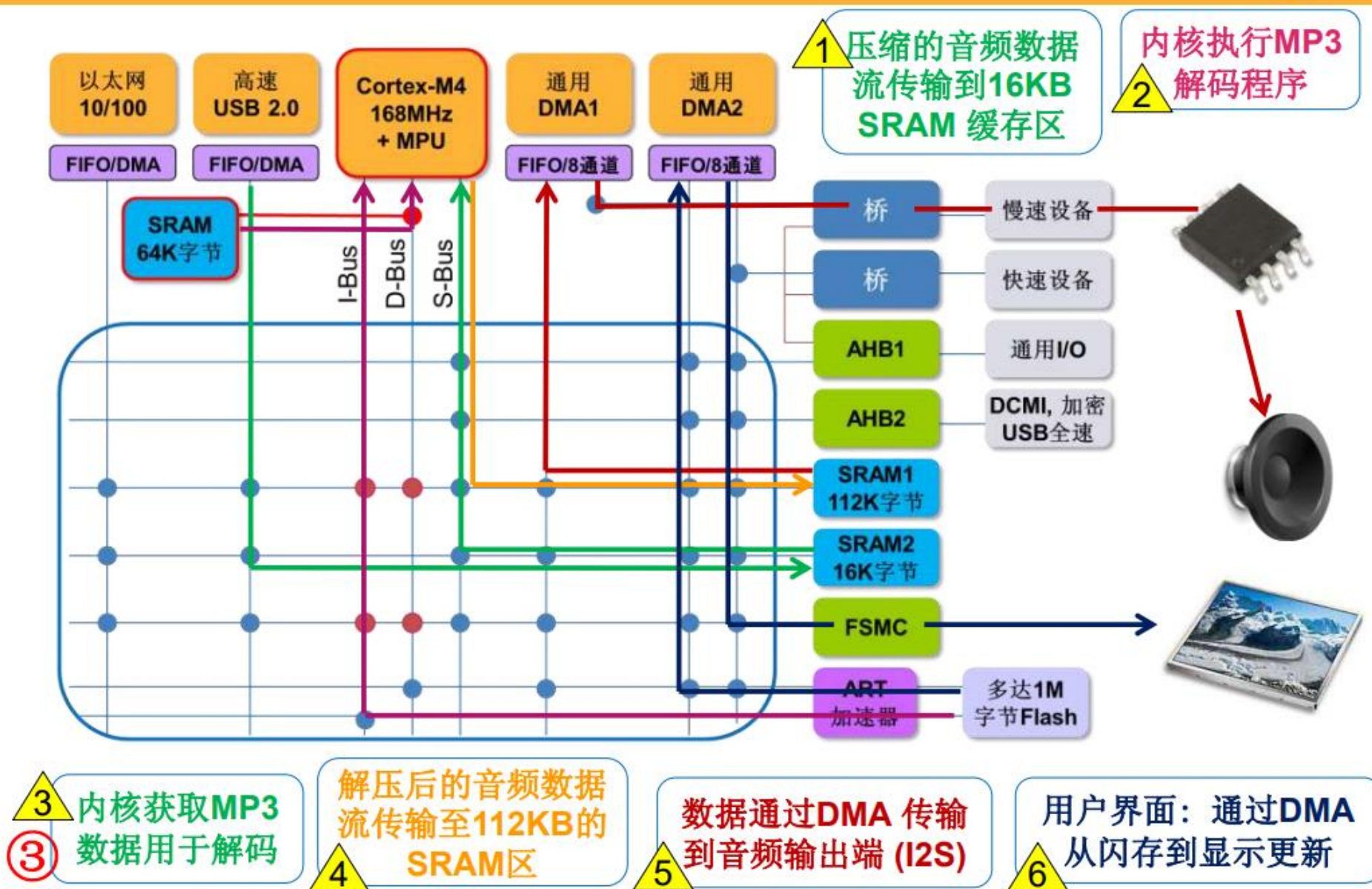
➤ D 总线

此总线用于将 Cortex™-M4F 数据总线和 64 KB CCM 数据 RAM 连接到总线矩阵。内核通过此总线进行立即数加载和调试访问。此总线访问的对象是包含代码或数据的存储器（内部Flash 或通过 FSMC 的外部存储器）。

➤ S 总线

此总线用于将 Cortex™-M4F 内核的系统总线连接到总线矩阵。此总线用于访问位于外设或 SRAM 中的数据。也可通过此总线获取指令（效率低于 ICode）。此总线访问的对象是112 KB、64 KB 和 16 KB 的内部 SRAM、包括 APB 外设在内的 AHB1 外设、AHB2 外设以及通过 FSMC 的外部存储器。

多重总线的并行处理能力



STM32F40x/41x框图



1. 高速功能需通过ULPI接口连接一个外部PHY 2. 只适用于STM32F417x和STM32F415x

STM32F407ZGT6 资源描述

◆内核:

- 32位 高性能ARM Cortex-M4处理器
- 时钟: 高达168M,实际还可以超屏一点点
- 支持FPU (浮点运算) 和DSP指令

◆IO口:

STM32F407ZGT6: 144引脚 114个IO

- 大部分IO口都耐5V(模拟通道除外)
- 支持调试: SWD和JTAG, SWD只要2根数据线

◆存储器容量: 1024K FLASH, 192K SRAM



◆AD:

- 3个12位AD【多达24个外部测量通道】
- 内部通道可以用于内部温度测量
- 内置参考电压

◆DA:

2个12位DA

◆DMA:

16个DMA通道，带FIFO和突发支持

支持外设：定时器，ADC,DAC，SDIO,I2S,SPI,I2C,和USART

◆定时器：多达17个定时器

-10个通用定时器（TIM2和TIM5是32位）

-2个基本定时器

-2个高级定时器

-1个系统定时器

-2个看门狗定时器



◆通信接口：多达17个通信接口

-3个I2C接口

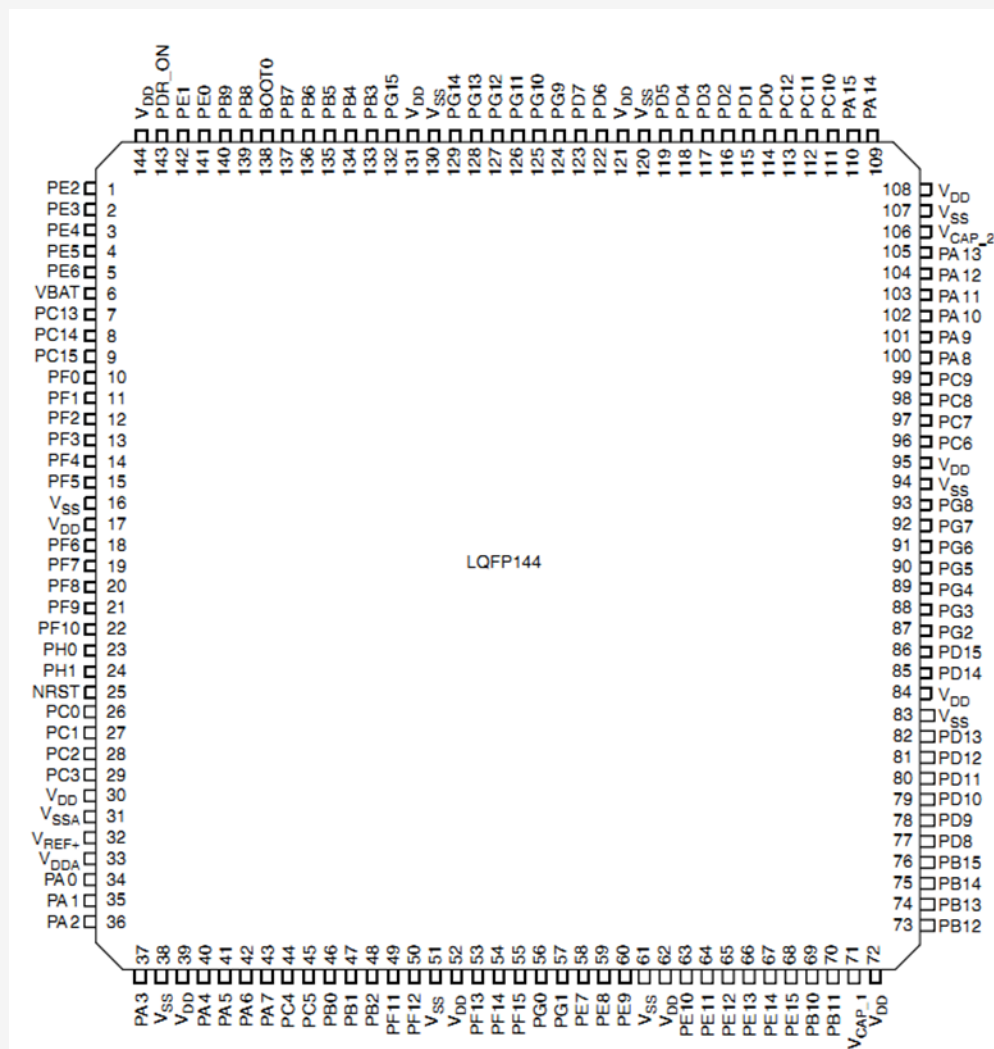
-6个串口

-3个SPI接口

-2个CAN2.0

-2个USB OTG

-1个SDIO



◆时钟，复位和电源管理：

① **1.8~3.6V**电源和**IO**电压

② 上电复位，掉电复位和可编程的电压监控

③ 强大的时钟系统

- **4~26M**的外部高速晶振

- 内部**16MHz**的高速**RC**振荡器

- 内部**32KHz**低速**RC**振荡器，看门狗时钟

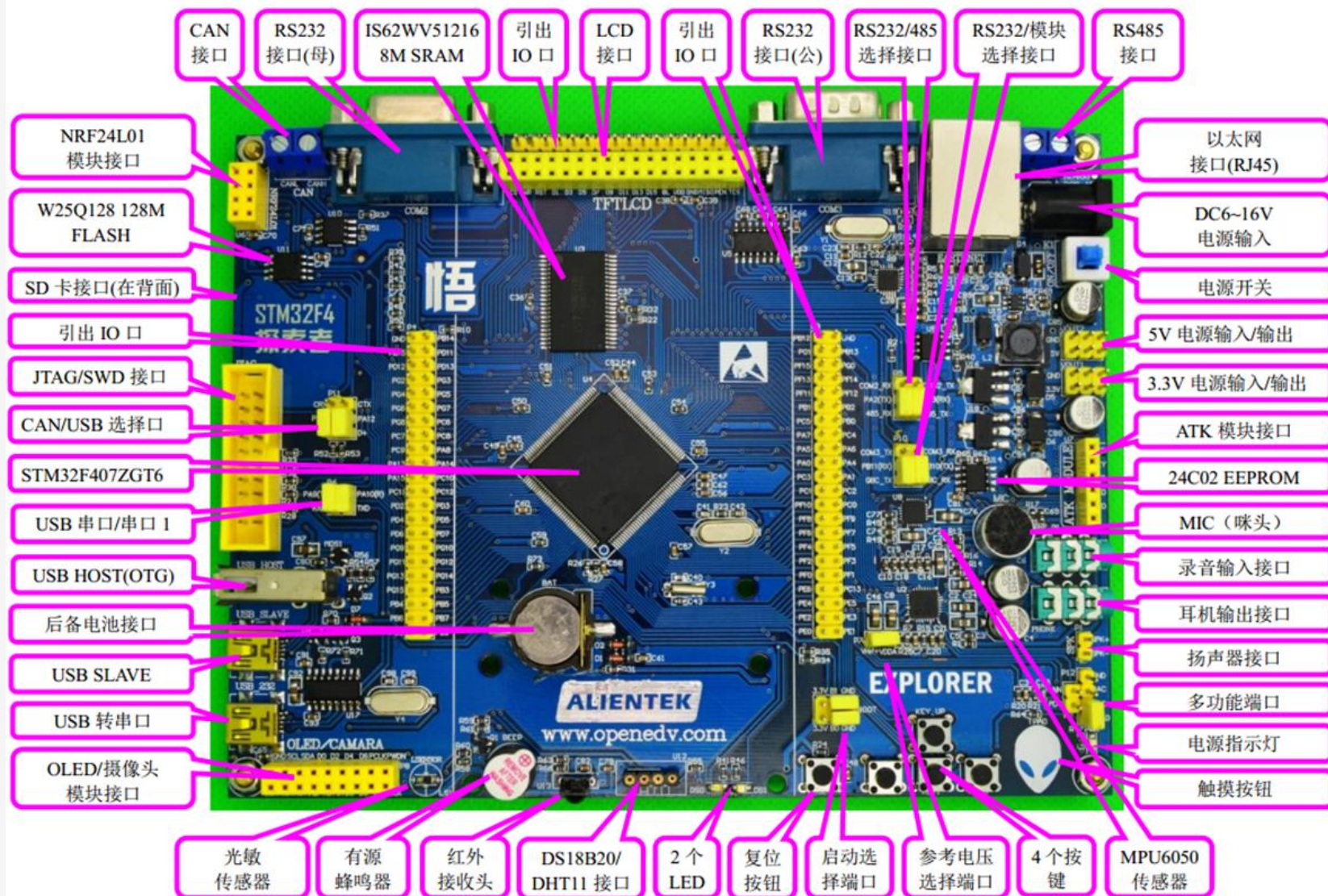
- 内部锁相环（**PLL**，倍频），一般系统时钟都是外部或者内部高速时钟经过**PLL**倍频后得到

- 外部低速**32.768K**的晶振，主要做**RTC**时钟源

STM32最小系统

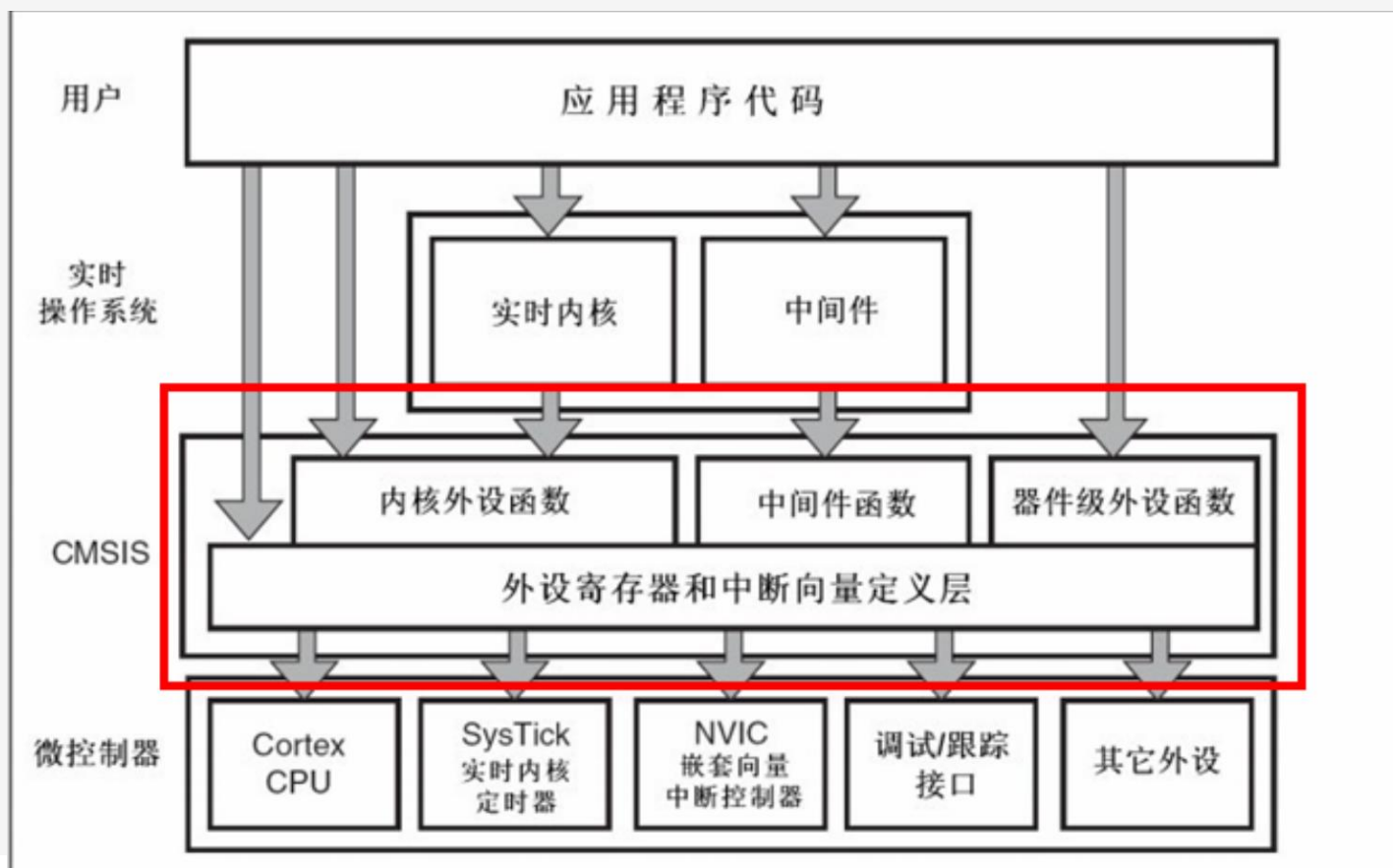
- 供电
- 复位
- 时钟：外部晶振（2个）
- Boot启动模式选择
- 下载电路（串口/JTAG/SWD)
- 后备电池

开发板外观说明



STM32 固件库与 CMSIS 标准

ARM 公司为了能让不同的芯片公司生产的 Cortex-M4 芯片能在软件上基本兼容，和芯片生产商共同提出了一套标准 **CMSIS** 标准(*Cortex Microcontroller Software Interface Standard*)，翻译过来是“ARM Cortex™ 微控制器软件接口标准”。ST 官方库就是根据这套标准设计的。



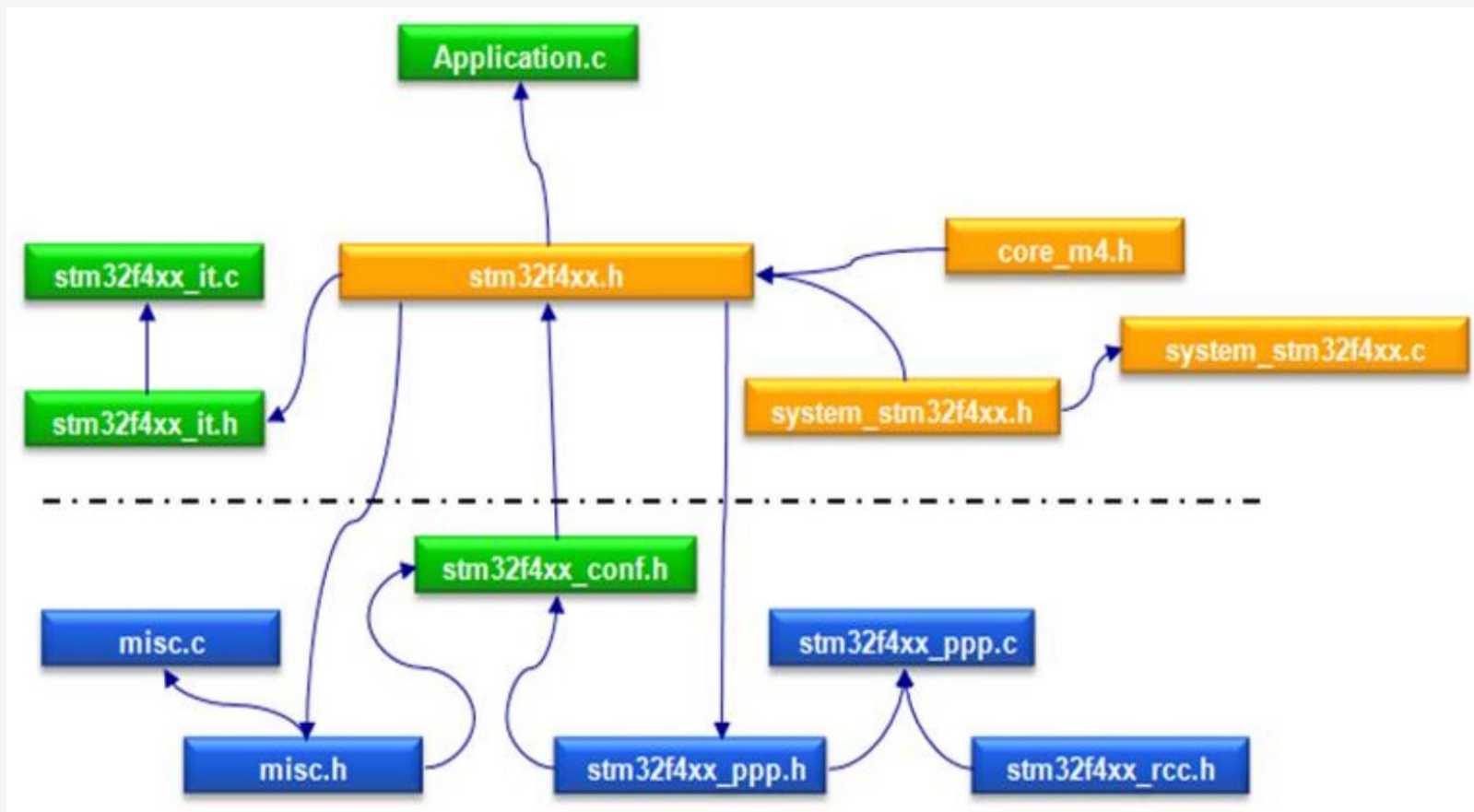
➤ CMSIS 分为 3 个基本功能层：

- 1) 核内外设访问层：ARM 公司提供的访问，定义处理器内部寄存器地址以及功能函数。
- 2) 中间件访问层：定义访问中间件的通用 API。由 ARM 提供，芯片厂商根据需要更新。
- 3) 外设访问层：定义硬件寄存器的地址以及外设的访问函数。

从图中可以看出，CMSIS 层在整个系统中是处于中间层，向下负责与内核和各个外设直接打交道，向上提供实时操作系统用户程序调用的函数接口。

- 如果没有 CMSIS 标准，那么各个芯片公司就会设计自己喜欢的风格的库函数，而 CMSIS 标准就是要强制规定，芯片生产公司设计的库函数必须按照 CMSIS 这套规范来设计。
- ✓ 一个简单的例子，我们在使用 STM32 芯片的时候首先要进行系统初始化，CMSIS 规范就规定，系统初始化函数名字必须为 `SystemInit`，所以各个芯片公司写自己的库函数的时候就必须用 `SystemInit` 对系统进行初始化。
- CMSIS 还对各个外设驱动文件的文件名字规范化，以及函数名字规范化等等一系列规定。比如函数 `GPIO_ResetBits` 这个函数名字也是不能随便定义的，是要遵循 CMSIS 规范的。

STM32F4 标准外设固件库文件



- core_cm4.h 文件是 CMSIS 核心文件，提供进入 M4 内核接口，这是 ARM 公司提供，对所有CM4 内核的芯片都一样。你永远都不需要修改这个文件。

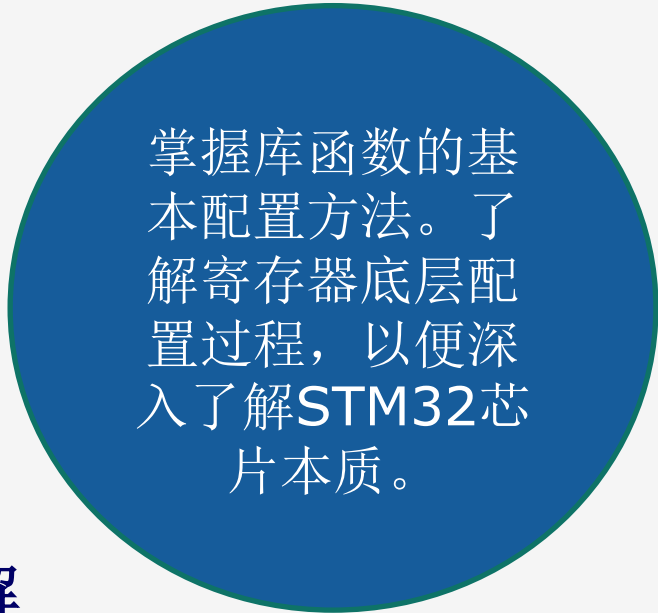
-
- ① `system_stm32f4xx.h` 是片上外设接入层系统头文件，主要是申明设置系统及总线时钟相关的函数。
 - ② `stm32f4xx.h` 是 STM32F4 片上外设访问层头文件，这个文件就相当重要了，只要你做 STM32F4 开发，你几乎时刻都要查看这个文件相关的定义。这个文件打开可以看到，里面非常多的结构体以及宏定义。这个文件里面主要是系统寄存器定义申明以及包装内存操作，对于这里是怎样申明以及怎样将内存操作封装起来的。同时该文件还包含了一些时钟相关的定义，FPU 和 MPU 单元开启定义，中断相关定义等等。
 - ③ `stm32f4xx_it.c` 和 `stm32f4xx_it.h` 里面是用来编写中断服务函数，中断服务函数也可以随意编写在工程里面的任意一个文件里面，
 - ④ `stm32f4xx_conf.h` 是外设驱动配置文件。文件打开可以看到一堆的 `#include`，这里你建立工程 的时候，可以注释掉一些你不用的外设头文件。
 - ⑤ `misc.c` 和 `misc.h` 是定义中断优先级分组以及 SysTick 定时 器相关的函数。
 - ⑥ `stm32f3xx_rcc.c` 和 `stm32f4xx_rcc.h` 是与 RCC 相关的一些操作函数，作用主要 是一些时钟的配置和使能。在任何一个 STM32 工程 RCC 相关的源文件和头文件是必须添加的。
 - ⑦ `stm32f4xx_ppp.c` 和 `stm32f4xx_ppp.h`，这就是 stm32F4 标准外设固件库对应的源 文件和头文件。包括一些常用外设 GPIO,ADC,USART 等。

熟练掌握一种开发环境

◆ 库函数和寄存器对比学习。

项目中大多数用库函数。
但是学习，如果你只会看几个函数的话，你根本没有学懂，遇到问题很难自己解决，所以必要了解一下寄存器配置原理，加深理解。

◆ 尤其前面几个章节实验，最好了解寄存器配置，加深对STM32本质的理解。



掌握库函数的基本配置方法。了解寄存器底层配置过程，以便深入了解STM32芯片本质。

◆库函数和寄存器的区别？

本质上是一样的。我们可以在库函数模板里面，直接操作寄存器，因为官方库相关头文件有寄存器定义。但是不能在寄存器模板调用库函数，因为没有引入库函数相关定义。

了解寄存器基本原理的目的是为了让我们对STM32相关知识有比较深入的理解，这样在开发过程中方可得心应手，游刃有余。底层代码配置出了问题需要调试的话，必须对寄存器有一定的了解才能找到问题，因为调试代码，底层只能查看寄存器相关配置。

➤ STM32的软件开发包

✓ STM32的固件库

ST公司目前提供了三种不同的库。

- 1、SPL（Standard Peripheral Libraries）库（固件库/标准库/STD库）
- 2、HAL（Hardware Abstraction Layer）库
- 3、LL（Low Layer）库

✓ STM32的固件库

(1) SPL库

SPL库与CMSIS标准兼容，所有的驱动源代码都符合“Strict ANSI-C”标准，程序源文件都按照Doxygen格式书写，使用这种书写格式的代码能够生成更加规范且内在关联性更强的程序说明文档。SPL库较好地实现了效率优化，无需直接操作寄存器。当然，SPL库也存在不足，由于每个SPL库的驱动包是针对特定的处理器系列，这就导致了不同系列处理器之间的可移植性并不好，因此ST公司已经不再为L0、L4、F4等新的STM32系列处理器提供SPL库驱动了。

✓ **STM32的固件库**

(2) HAL库

HAL（Hardware Abstraction Layer）库是ST为STM32提供嵌入式中间件，用来取代之前SPL，可以更好的确保跨STM32产品的最大可移植性，且基于BSD许可协议开放源代码。HAL库的API集中关注各外设的公共函数功能，这样便于定义一套通用的用户友好的API函数接口，从而可以轻松实现从一个STM32产品移植到另一个不同的STM32系列产品。

HAL库是目前ST公司主推用户使用的库，它较好的实现了硬件抽象化，有很好的RTOS兼容性，能够节约开发时间，同时程序可移植性较强。HAL库的缺点在于其编译后的代码较SPL占用更多的存储空间，程序执行效率不如SPL，不适合Cortex-M0/L0这类低端的应用处理器。

✓ STM32的固件库

(3) LL库

为了编程能够生成更精炼的代码，提高程序的执行效率，ST公司提供了LL（Low Layer）库，LL库更接近硬件层，提供寄存器级别的编程。LL库根据各处理器产品线的技术文档提供了STM32外部设备的服务程序，这些服务程序准确的反映了硬件的功能，提供了访问这些外设的原子操作。LL库提供的服务程序不是基于独立的进程，所以不需要任何额外的资源来保存处理器的状态、计数器或数据指针，因此有较高的执行效率。但LL库依赖于特定的STM32处理器，不同的处理器系列之间无法直接共享代码，不支持功能复杂的外设如USB、FSMC、SDMMC等，用户使用LL库是需要了解各个外设寄存器的功能细节。

LL库可以独立使用，也可以和HAL库结合使用。

✓ STM32的固件库

- 用一个简单的例子对比了SPL、HAL和LL的差别

SPL库代码

```
GPIO_ResetBits(GPIOF,GPIO_Pin_9);  
GPIO_SetBits(GPIOF,GPIO_Pin_10);
```

HAL库代码

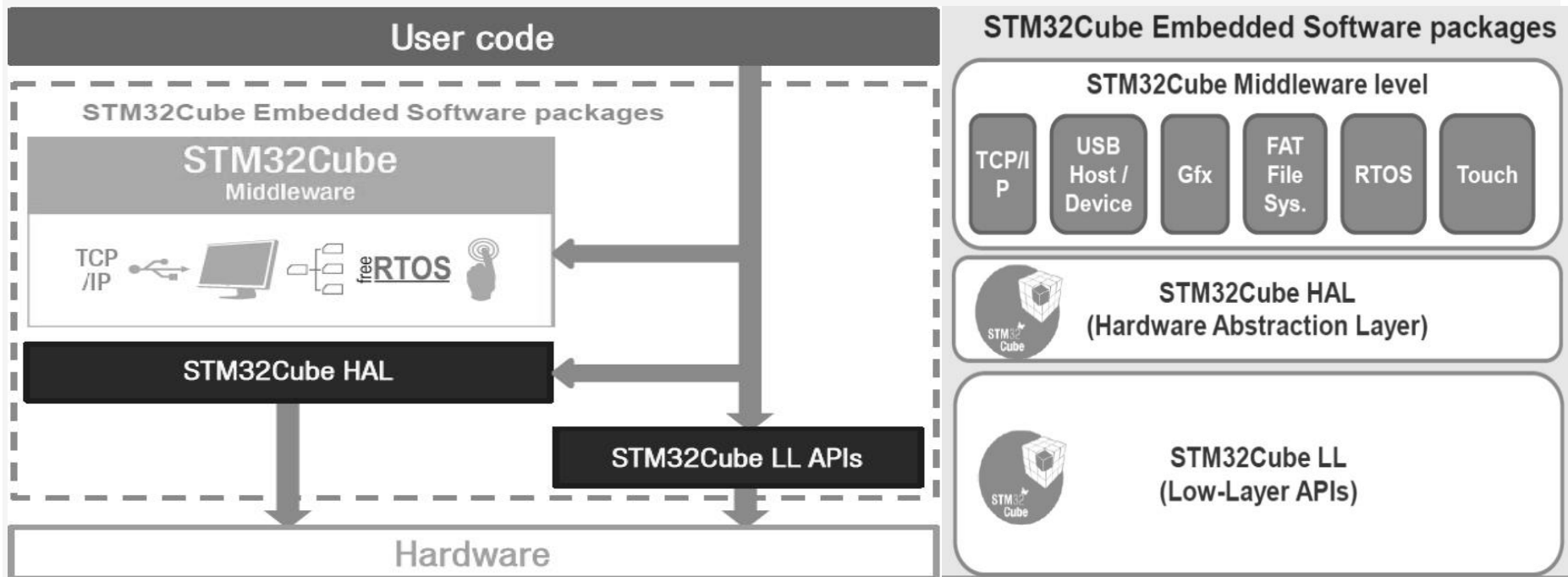
```
HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, GPIO_PIN_RESET);  
HAL_GPIO_WritePin(GPIOF, GPIO_Pin_10, GPIO_PIN_SET);
```

LL库代码

```
LL_GPIO_ResetOutputPin(GPIOF, GPIO_PIN_9,);  
LL_GPIO_SetOutputPin(GPIOF, GPIO_Pin_10);
```

➤ STM32CubeMX介绍

STM32CubeMX是ST公司提供一款辅助开发工具，它通过图形化的方式来配置处理器各个部件的功能，自动生成初始化和配置代码，从而节约开发时间，提高开发效率。



STM32CubeMX结构简图

MDK5 简介

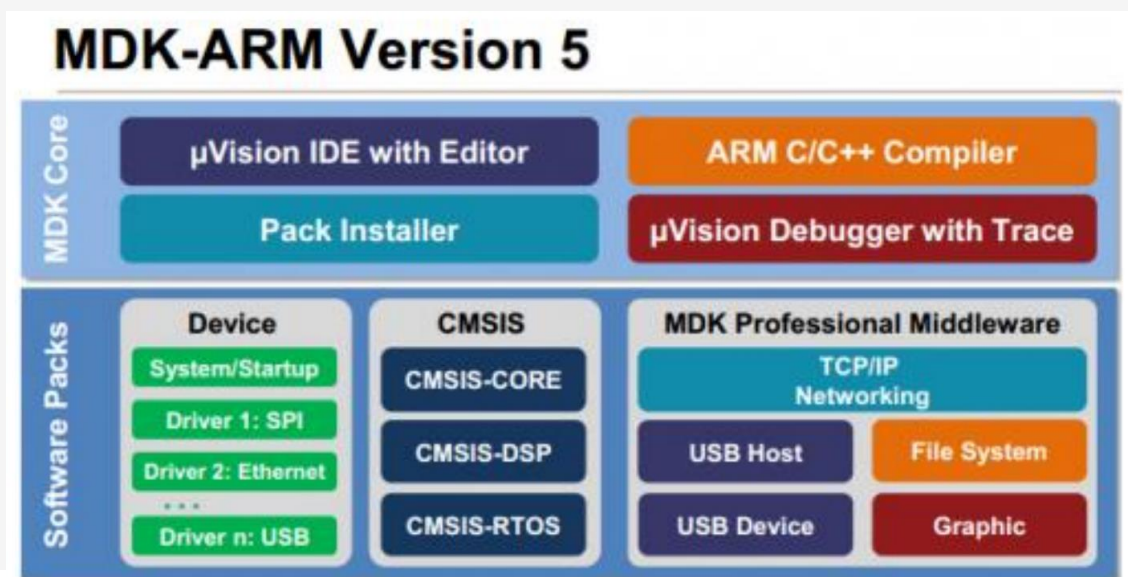
- MDK 源自德国的 KEIL 公司，是 RealView MDK 的简称。在全球 MDK 被超过 10 万的嵌入式开发工程师使用。
- 目前最新版本为：MDK5.14，该版本使用 uVision5 IDE 集成开发环境，是目前针对 ARM 处理器，尤其是 Cortex M 内核处理器的最佳开发工具。
- MDK5 向后兼容 MDK4 和 MDK3 等，以前的项目同样可以在 MDK5 上进行开发(但是头文件方面得全部自己添加)，MDK5 同时加强了针对 Cortex-M 微控制器开发的支持，并且对传统的开发模式和界面进行升级。
- MDK5 由两个部分组成：MDK Core 和 Software Packs。其中，Software Packs 可以独立于工具链进行新芯片支持和中间库的升级。

MDK5 简介

MDK Core 又分成四个部分：uVision IDE with Editor（编辑器），ARM C/C++ Compiler（编译器），Pack Installer（包安装器），uVision Debugger with Trace（调试跟踪器）。

uVision IDE 从 MDK4.7 版本开始就加入了代码提示功能和语法动态检测等实用功能，相对于以往的 IDE 改进很大。

Software Packs（包安装器）又分为：Device（芯片支持），CMSIS（ARM Cortex 微控制器软件接口标准）和Mddleware（中间库）三个小部分，通过包安装器，我们可以安装最新的组件，从而支持新的器件、提供新的设备驱动库以及最新例程等。



从基本外设开始，到高级功能的学习

✓ 基本外设：

- GPIO输入输出，外部中断，定时器，**串口**。
- 理解了这四个外设，基本就入门了一款MCU。

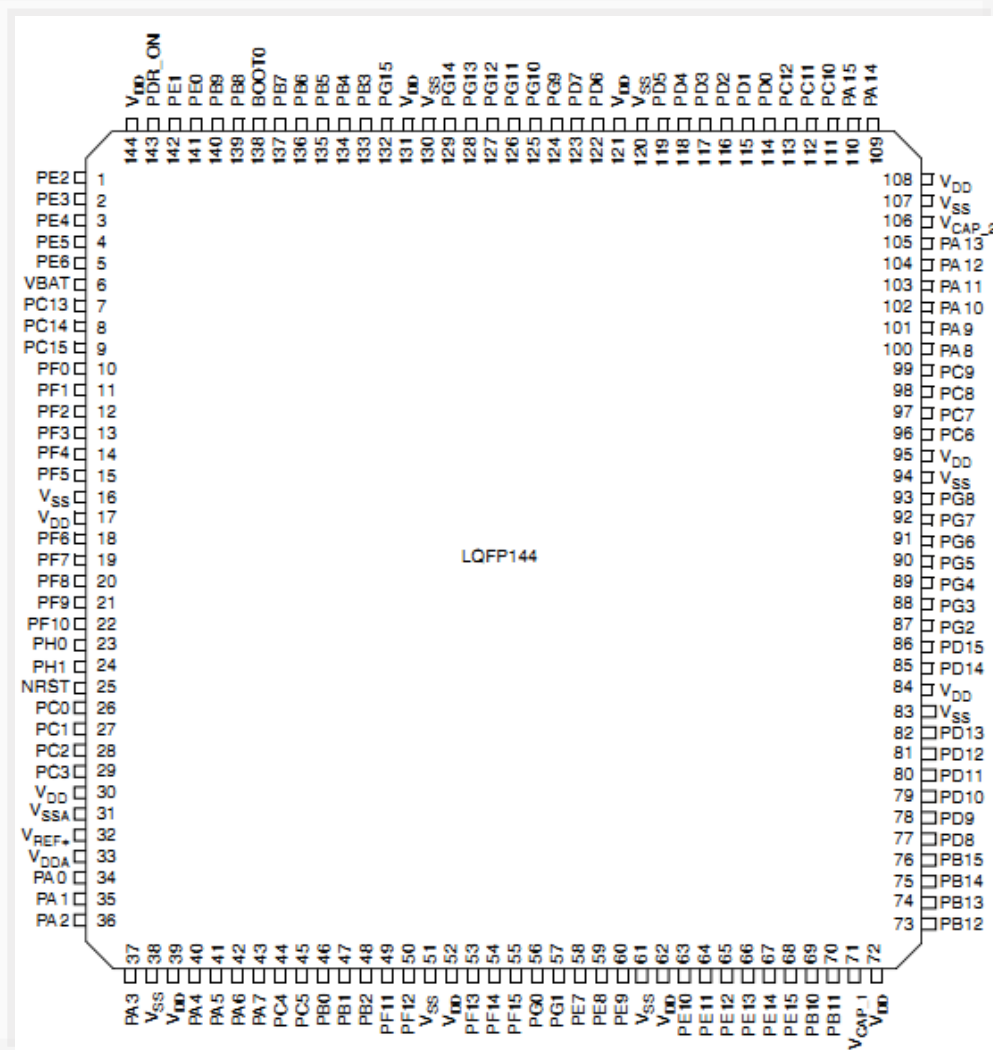
✓ 基本外设接口：

- SPI, IIC, WDG, FSMC, ADC/DAC, SDIO等
- 这些外设接口功能原理对每个芯片几乎都是一样。**
对芯片而言就是加减法而已。

✓ 高级功能：

- UCOS, FATFS, EMWIN等。以及一些应用。

✓ 1.3 STM32引脚说明



◆ STM32F407ZGT6

- 一共有7组IO口
- 每组IO口有16个IO
- 一共 $16 \times 7 = 112$ 个IO

外加2个PH0和PH1
一共114个IO口

**GPIOA, GPIOB---GPIOG
PH0, PH1**

✓ GPIO基本结构

42	68	101	E15	120	PA9	I/O	FT	USART1_TX/ TIM1_CH2 / I2C3_SMBA / DCMI_D0/ EVENTOUT	OTG_FS_VBUS
43	69	102	D15	121	PA10	I/O	FT	USART1_RX/ TIM1_CH3/ OTG_FS_ID/DCMI_D1/ EVENTOUT	
44	70	103	C15	122	PA11	I/O	FT	USART1_CTS / CAN1_RX / TIM1_CH4 / OTG_FS_DM/ EVENTOUT	
45	71	104	B15	123	PA12	I/O	FT	USART1_RTS / CAN1_TX/ TIM1_ETR/ OTG_FS_DP/ EVENTOUT	

◆ STM32的大部分引脚除了当GPIO使用外，还可以复用为外设功能引脚（比如串口）。

✓ 1.1 GPIO工作方式

◆ 4种输入模式:

- 输入浮空
- 输入上拉
- 输入下拉
- 模拟输入

◆ 4种输出模式:

- 开漏输出(带上拉或者下拉)
- 开漏复用功能(带上拉或者下拉)
- 推挽式输出(带上拉或者下拉)
- 推挽式复用功能(带上拉或者下拉)

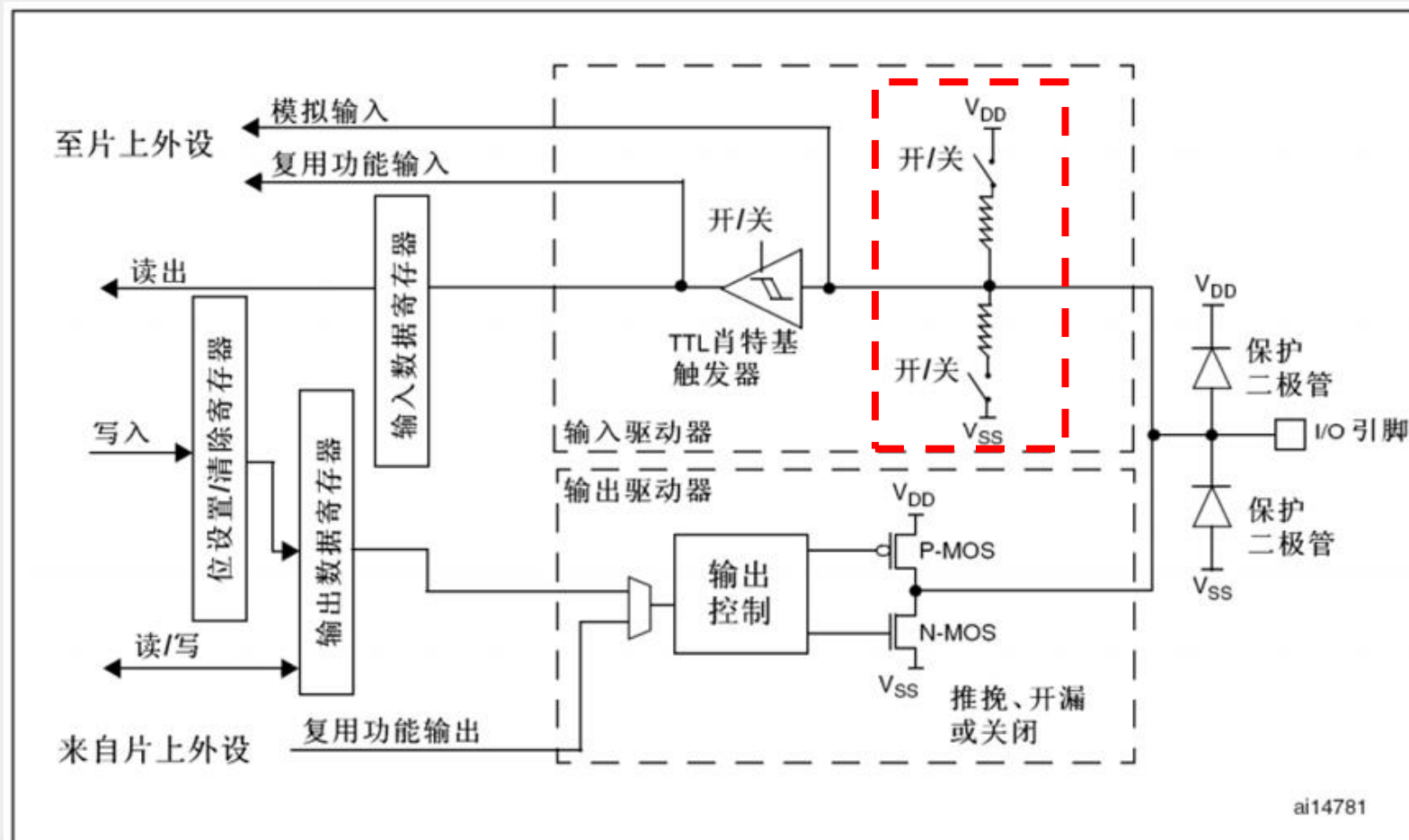
◆ 4种最大输出速度:

- 2MHZ
- 25MHz
- 50MHz
- 100MHz

8种工作模式的区别: STM32八种IO口模式区别.pdf

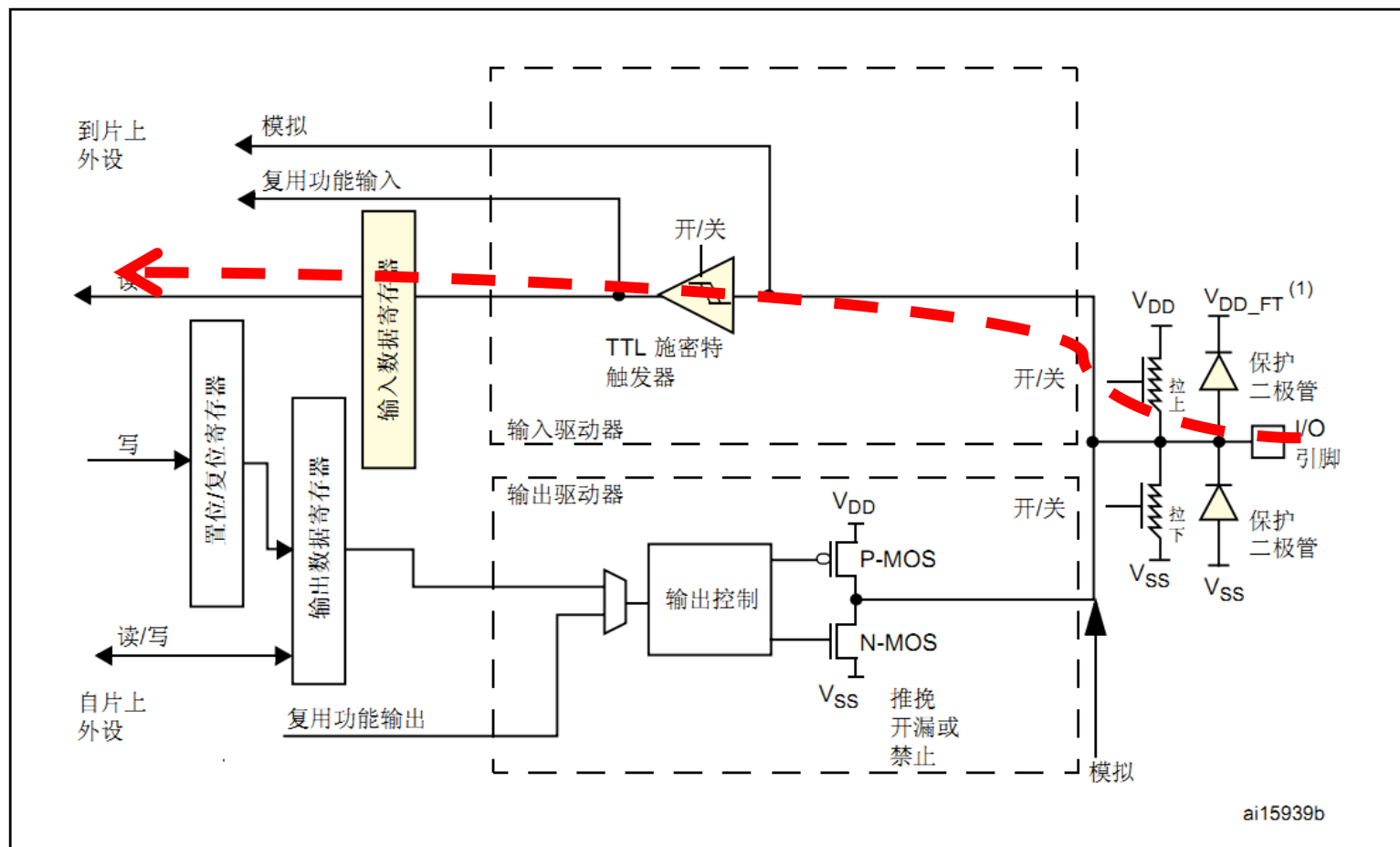
✓ 1.1 GPIO基本结构

M3的IO口基本结构



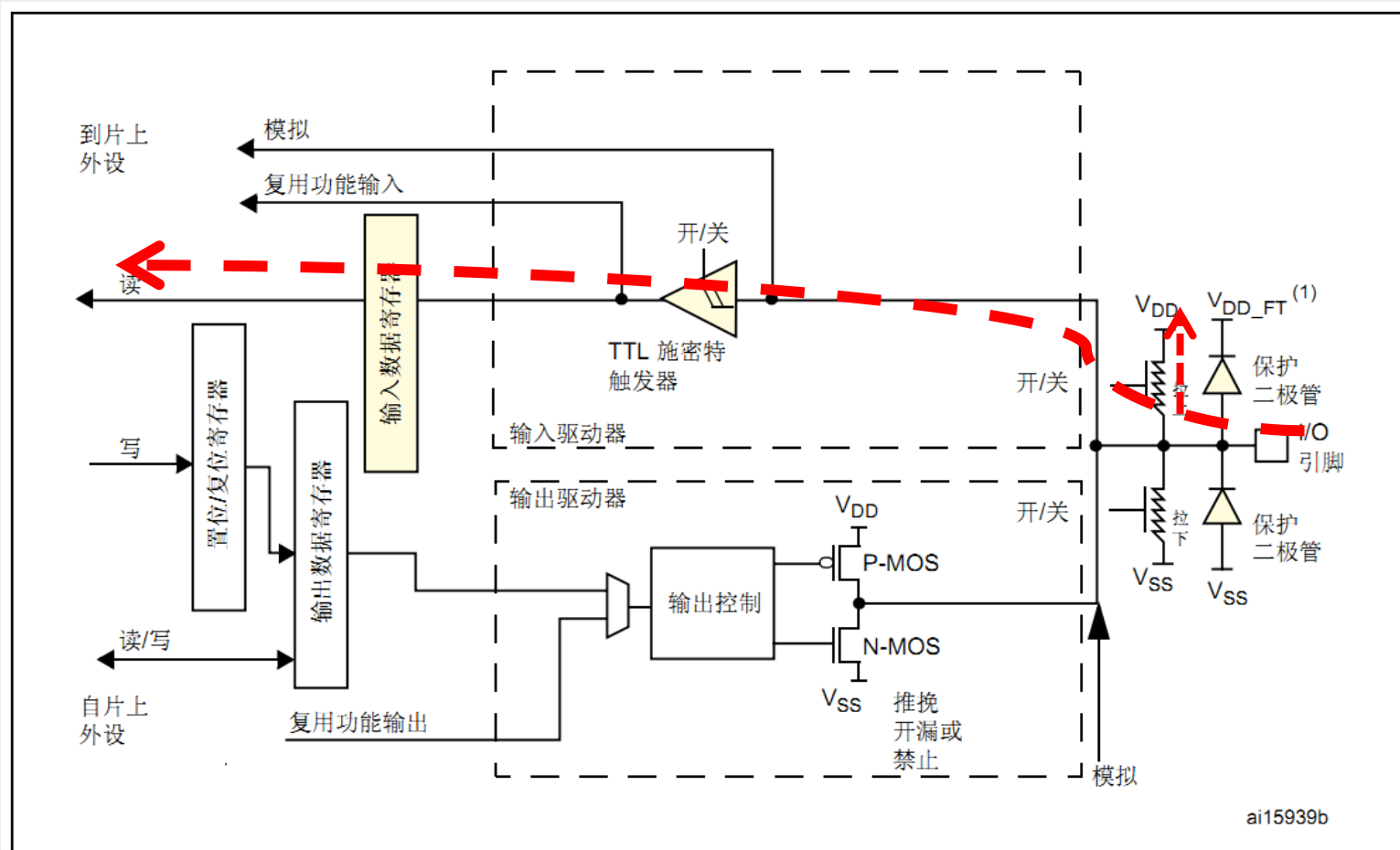
✓ 1.1 GPIO基本结构

◆ GPIO的输入工作模式1—输入浮空模式



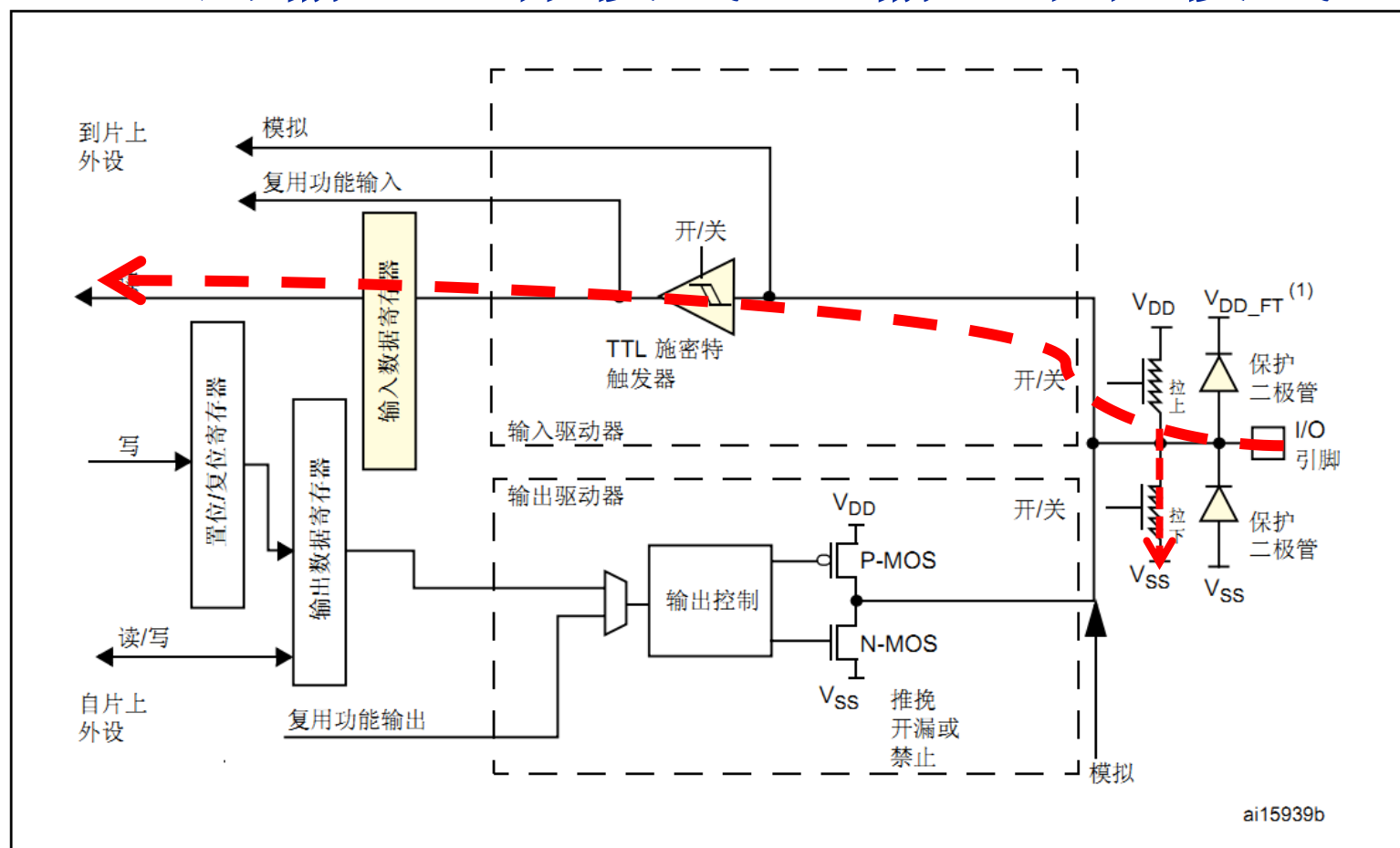
✓ 1.1 GPIO基本结构

◆ GPIO的输入工作模式2—输入上拉模式



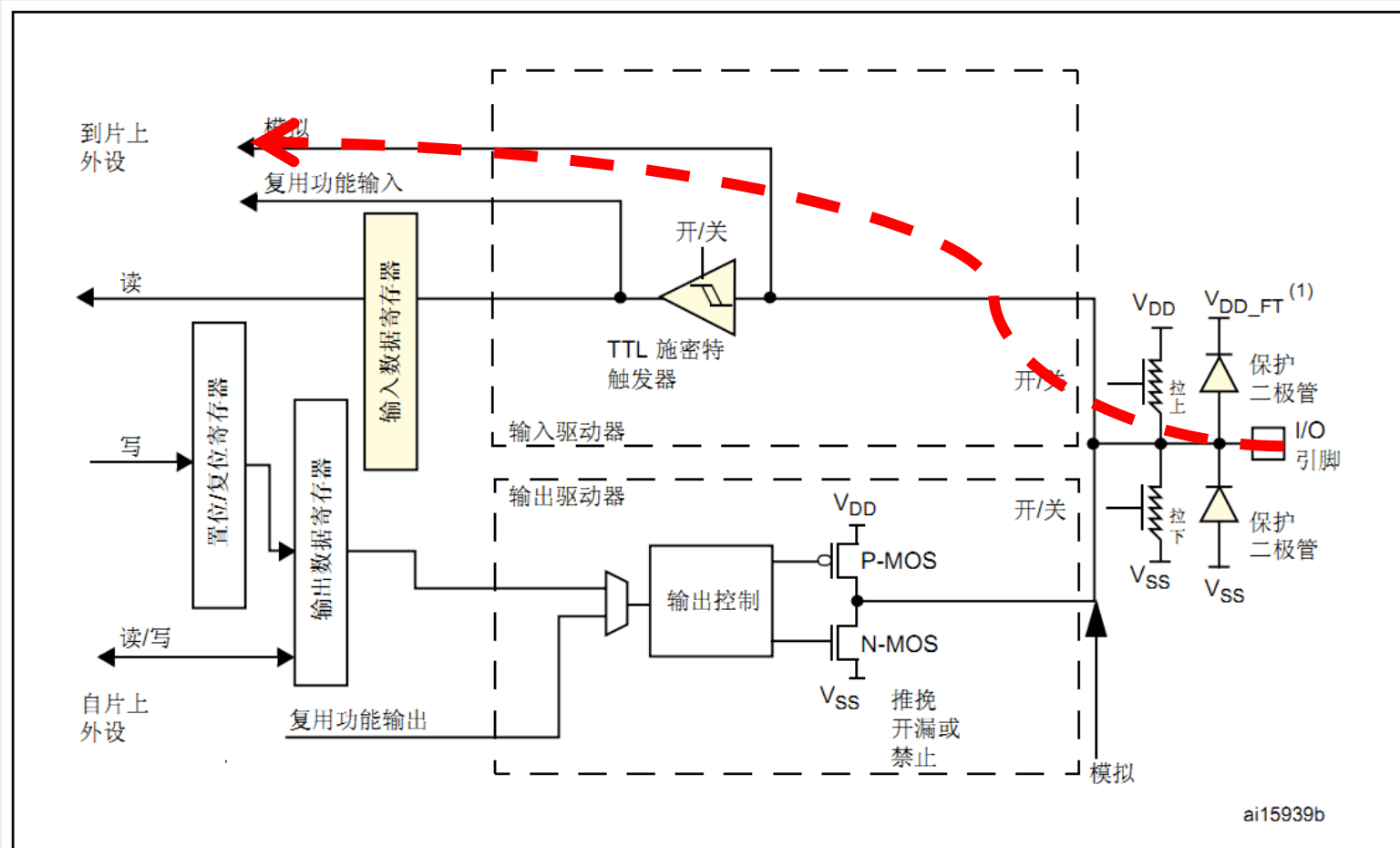
✓ 1.1 GPIO基本结构

◆ GPIO的输入工作模式3—输入下拉模式



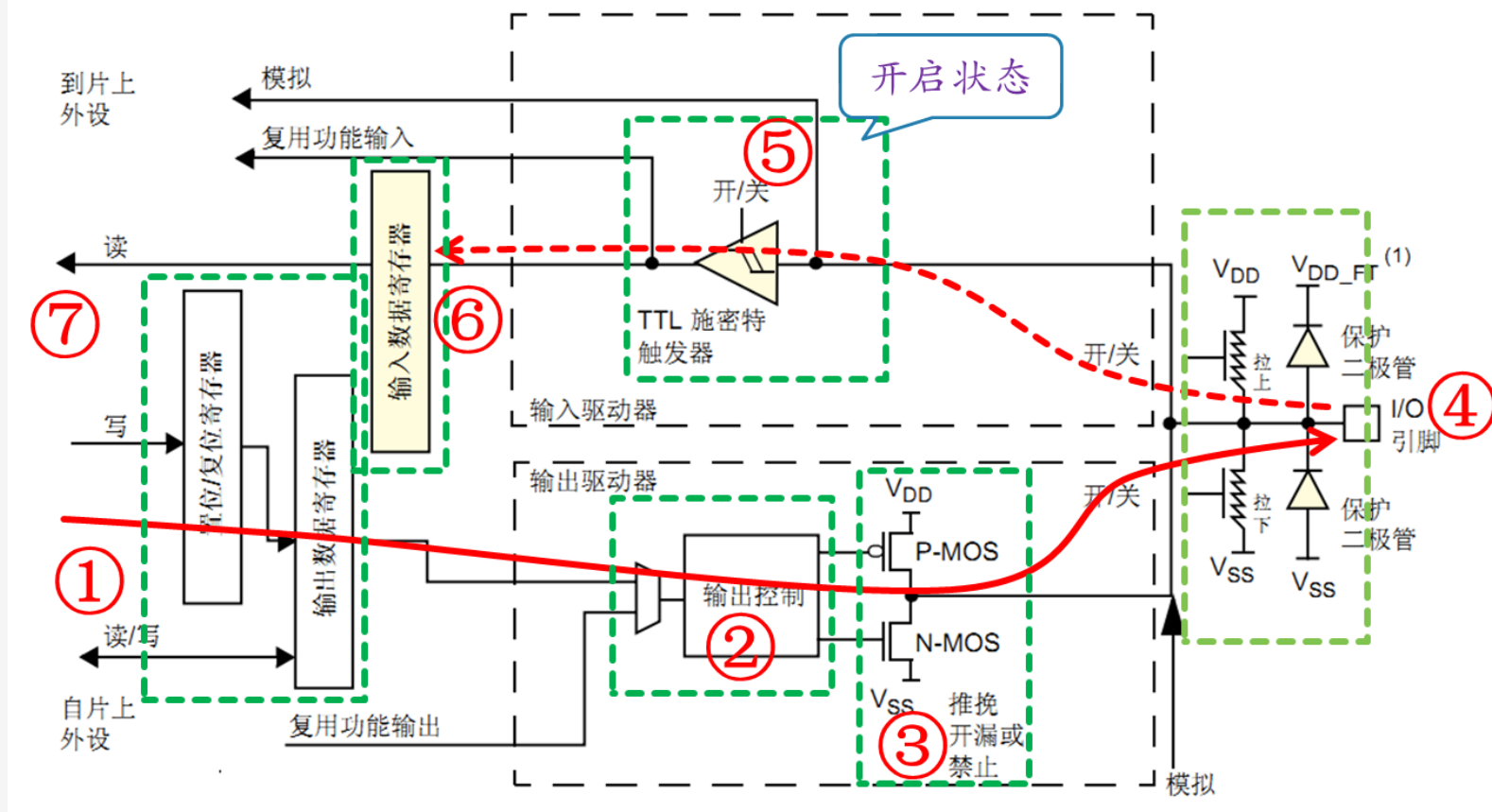
✓ 1.1 GPIO基本结构

◆ GPIO的输入工作模式4—模拟模式



✓ 1.1 GPIO工作方式

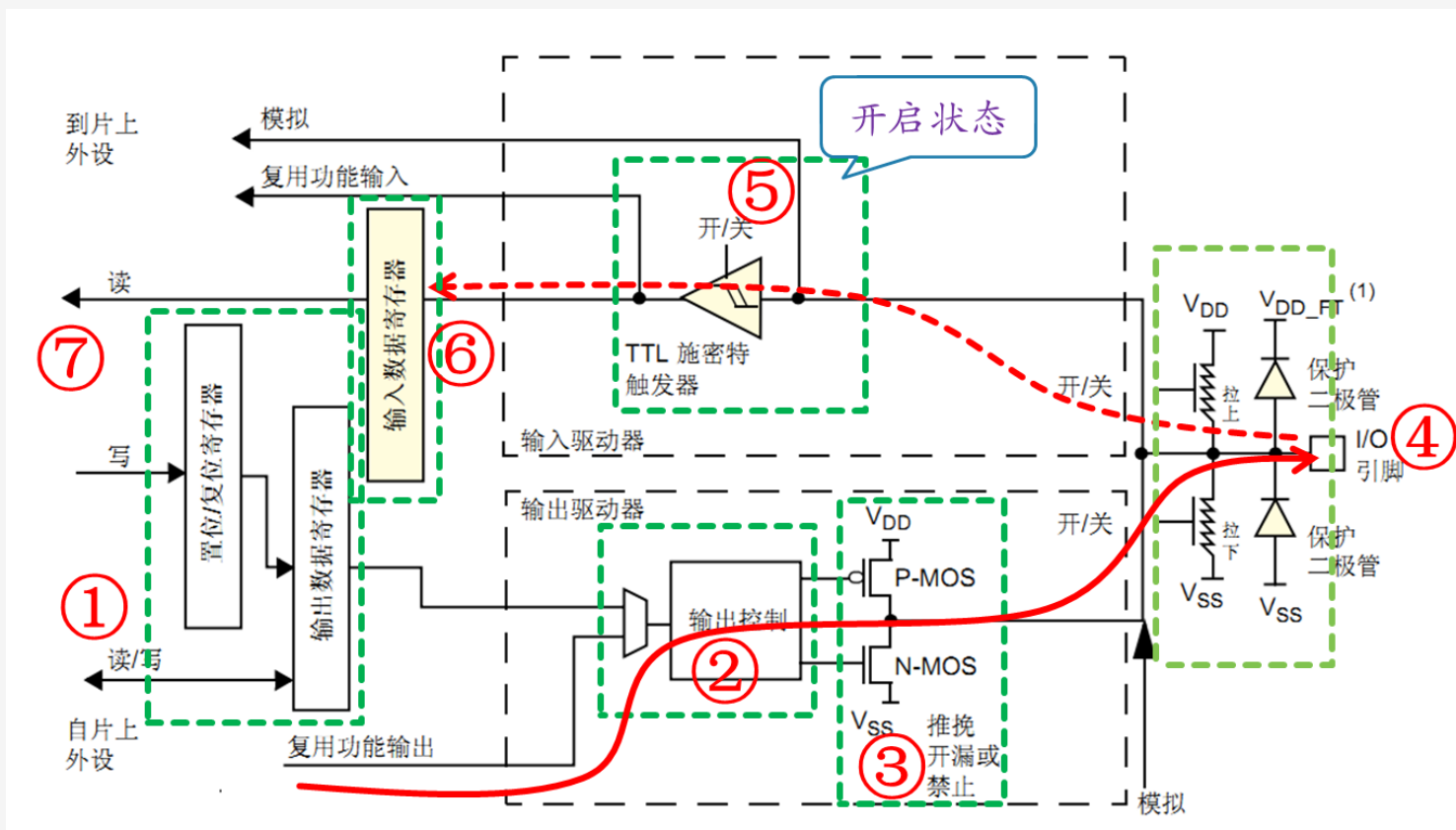
◆ GPIO的输出工作模式1—开漏输出模式



开漏模式下P-MOS不工作。输出高电平时，N-MOS截止；
输出低电平时，N-MOS导通

✓ 1.1 GPIO工作方式

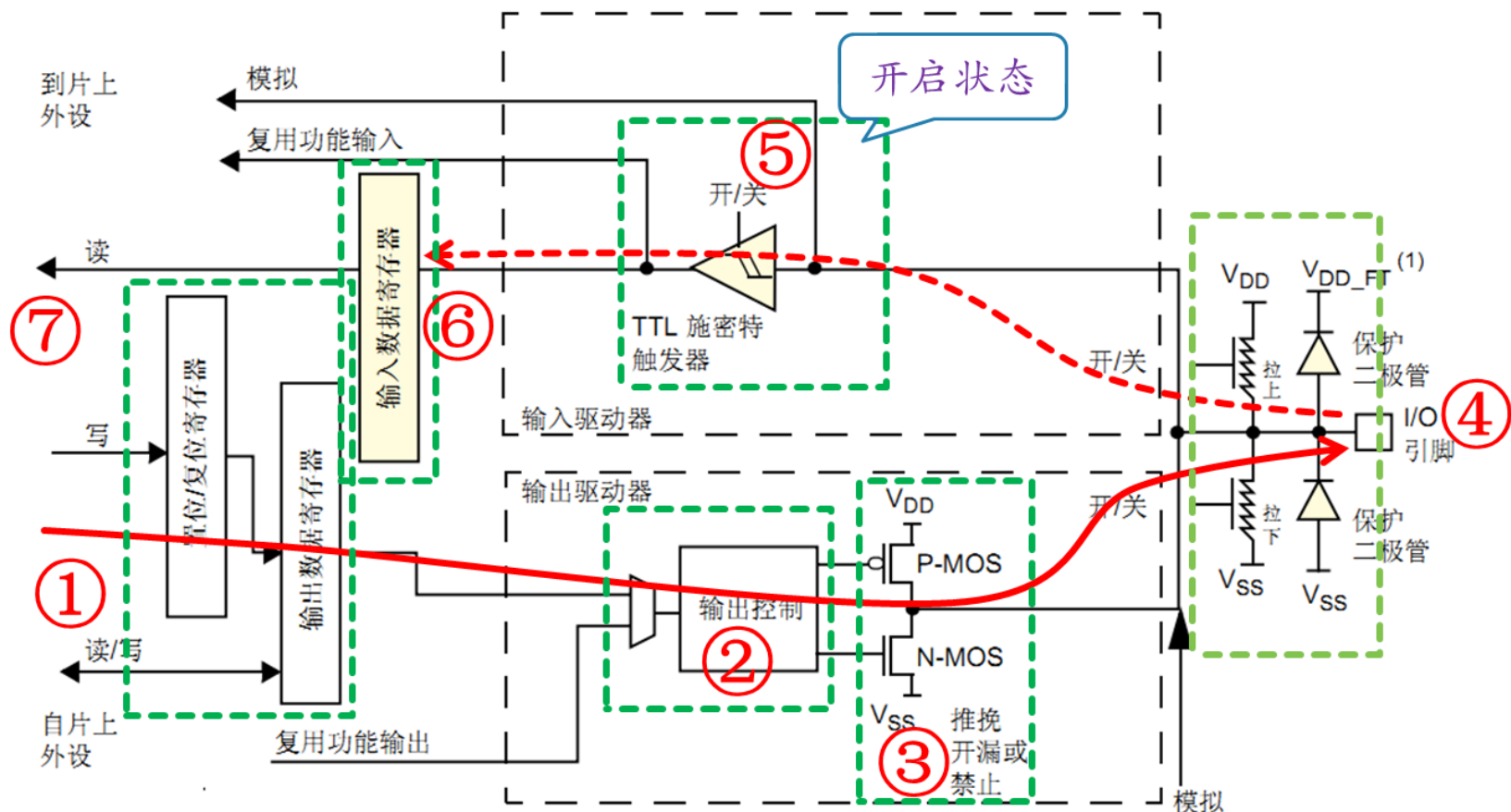
◆ GPIO的输出工作模式2—开漏复用输出模式



开漏模式下P-MOS不工作。输出高电平时，N-MOS截止；
输出低电平时，N-MOS导通

✓ 1.1 GPIO工作方式

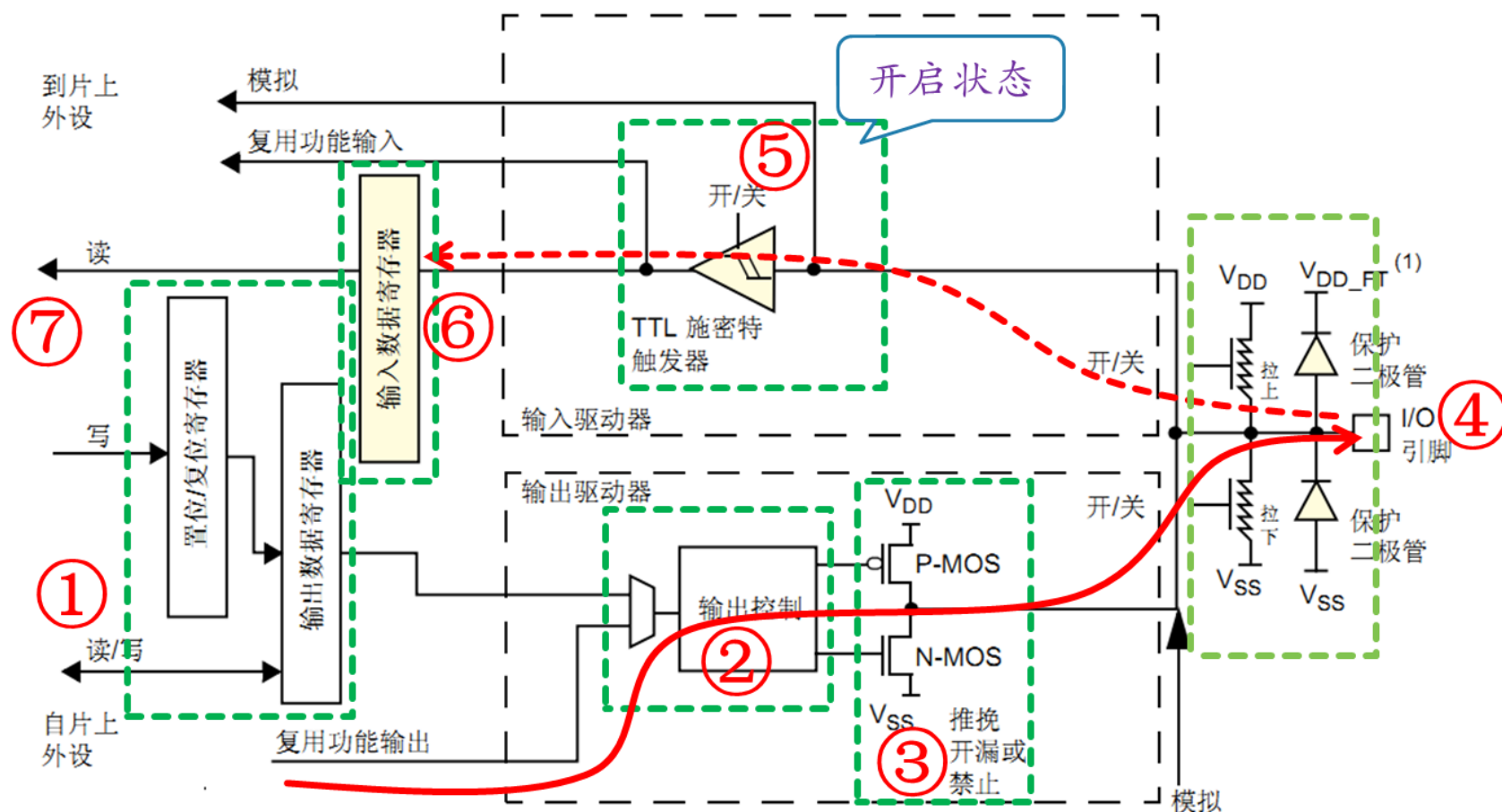
◆ GPIO的输出工作模式3—推挽输出模式



推挽模式下：输出高电平时，P-MOS导通，N-MOS截止；
输出低电平时，P-MOS截止，N-MOS导通

✓ 1.1 GPIO工作方式

◆ GPIO的输出工作模式4—推挽复用输出模式



✓ 1.1 GPIO工作方式

◆ 上电复位后，GPIO默认为浮空状态，部分特殊功能引脚为特定状态。

在复位期间及复位刚刚完成后，复用功能尚未激活，I/O 端口被配置为输入浮空模式。

复位后，调试引脚处于复用功能上拉/下拉状态：

- PA15: JTDI 处于上拉状态
- PA14: JTCK/SWCLK 处于下拉状态
- PA13: JTMS/SWDAT 处于下拉状态
- PB4: NJTRST 处于上拉状态
- PB3: JTDO 处于浮空状态

当引脚配置为输出后，写入到输出数据寄存器 (GPIOx_ODR) 的值将在 I/O 引脚上输出。可以在推挽模式下或开漏模式下使用输出驱动器（输出 0 时仅激活 N-MOS）。

输入数据寄存器 (GPIOx_IDR) 每隔 1 个 AHB1 时钟周期捕获一次 I/O 引脚的数据。

所有 GPIO 引脚都具有内部弱上拉及下拉电阻，可根据 GPIOx_PUPDR 寄存器中的值来打开/关闭。

✓ 1.1 GPIO工作方式

推挽输出:

可以输出强高低电平, 连接数字器件

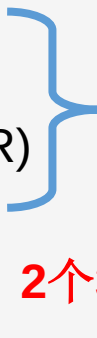
开漏输出:

只可以输出强低电平, 高电平得靠外部电阻拉高。输出端相当于三极管的集电极. 要得到高电平状态需要上拉电阻才行. 适合于做电流型的驱动,其吸收电流的能力相对强(一般20ma以内)

✓ 1.2 GPIO相关配置寄存器

每组GPIO端口的寄存器包括：

一个端口模式寄存器 (GPIOx_MODER)
一个端口输出类型寄存器(GPIOx_OTYPER)
一个端口输出速度寄存器 (GPIOx_OSPEEDR)
一个端口上拉下拉寄存器 (GPIOx_PUPDR)
一个端口输入数据寄存器 (GPIOx_IDR)
一个端口输出数据寄存器 (GPIOx_ODR)
一个端口置位/复位寄存器 (GPIOx_BSRR)
一个端口配置锁存寄存器 (GPIOx_LCKR)
两个复用功能寄存器 (低位GPIOx_AFRL & GPIOx_AFRH)

 4个32位配置寄存器
2个32位数据寄存器

- ◆ 如果配置一个IO口需要2个位，那么刚好32位寄存器配置一组IO口16个IO口
- ◆ 如果配置一个IO口只需要1个位，一般高16位保留
- ◆ BSRR寄存器32位分为低16位BSRRL和高16位BSRRH，BSRRL配置一组IO口的16个IO口的置位状态（1），BSRRH配置复位状态（0）。

✓ 1.2 GPIO相关配置寄存器

是每组IO口含下面10个寄存器。也就是10个寄存器，一共可以控制一组GPIO的16个IO口。

- 一个端口模式寄存器 (GPIOx_MODER)
- 一个端口输出类型寄存器(GPIOx_OTYPER)
- 一个端口输出速度寄存器 (GPIOx_OSPEEDR)
- 一个端口上拉下拉寄存器 (GPIOx_PUPDR)
- 一个端口输入数据寄存器 (GPIOx_IDR)
- 一个端口输出数据寄存器 (GPIOx_ODR)
- 一个端口置位/复位寄存器 (GPIOx_BSRR)
- 一个端口配置锁存寄存器 (GPIOx_LCKR)
- 两个复用功能寄存器 (低位GPIOx_AFRL & GPIOx_AFRH)

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.1 端口模式寄存器(GPIOx_MODER)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 $2y:2y+1$ **MODERy[1:0]**: 端口 x 配置位 (Port x configuration bits) ($y = 0..15$)

这些位通过软件写入，用于配置 I/O 方向模式。

00: 输入（复位状态）

01: 通用输出模式

10: 复用功能模式

11: 模拟模式

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.2 端口输出类型寄存器(GPIOx_OTYPER)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **OTy[1:0]**: 端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 端口的输出类型。

0: 输出推挽（复位状态）

1: 输出开漏

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.3 端口输出速度寄存器(GPIOx_OSPEEDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 $2y:2y+1$ **OSPEEDRy[1:0]**: 端口 x 配置位 (Port x configuration bits) ($y = 0..15$)

这些位通过软件写入，用于配置 I/O 输出速度。

00: 2 MHz (低速)

01: 25 MHz (中速)

10: 50 MHz (快速)

11: 30 pF 时为 100 MHz (高速) (15 pF 时为 80 MHz 输出 (最大速度))

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.4 端口上拉/下拉寄存器(GPIOx_PUPDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 $2y:2y+1$ **PUPDRy[1:0]**: 端口 x 配置位 (Port x configuration bits) ($y = 0..15$)

这些位通过软件写入，用于配置 I/O 上拉或下拉。

00: 无上拉或下拉

01: 上拉

10: 下拉

11: 保留

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.5 端口输入数据寄存器(GPIOx_IDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 31:16 保留，必须保持复位值。

位 15:0 **IDRy[15:0]**: 端口输入数据 (Port input data) ($y = 0..15$)

这些位为只读形式，只能在字模式下访问。它们包含相应 I/O 端口的输入值。

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.6 端口输出数据寄存器 (GPIOx_ODR) (x = A..I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **ODRy[15:0]**: 端口输出数据 (Port output data) (y = 0..15)

这些位可通过软件读取和写入。

注意: 对于原子置位/复位, 通过写入 **GPIOx_BSRR** 寄存器, 可分别对 **ODR** 位进行置位和复位 (x = A..I)。

✓ 1.2 GPIO相关配置寄存器

◆ 1.2.7 端口置位/复位寄存器(GPIOx_BSRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**: 端口 x 复位位 y (Port x reset bit y) (y = 0..15)

这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。

0: 不会对相应的 ODRx 位执行任何操作

1: 对相应的 ODRx 位进行复位

注意: 如果同时对 BSx 和 BRx 置位, 则 BSx 的优先级更高。

位 15:0 **BSy**: 端口 x 置位位 y (Port x set bit y) (y= 0..15)

这些位为只写形式，只能在字、半字或字节模式下访问。读取这些位可返回值 0x0000。

0: 不会对相应的 ODRx 位执行任何操作

1: 对相应的 ODRx 位进行置位

✓ 1. 端口复用

◆ 什么是端口复用？

STM32有很多的内置外设，这些外设的外部引脚都是与GPIO复用的。也就是说，一个GPIO如果可以复用为内置外设的功能引脚，那么当这个GPIO作为内置外设使用的时候，就叫做复用。

✓ 1. 端口复用

- ◆ 例如串口1 的发送接收引脚是PA9,PA10，当我们把PA9,PA10不用作GPIO，而用做复用功能串口1的发送接收引脚的时候，叫端口复用。

Pin number					Pin name (function after reset) ⁽¹⁾	Pin type	I / O structure	Notes	Alternate functions	Additional functions
LQFP64	LQFP100	LQFP144	UFBGA176	LQFP176						
41	67	100	F15	119	PA8	I/O	FT		MCO1 / USART1_CK/ TIM1_CH1/ I2C3_SCL/ OTG_FS_SOF/ EVENTOUT	
42	68	101	E15	120	PA9	I/O	FT		USART1_TX/ TIM1_CH2 / I2C3_SMBA / DCMI_D0/ EVENTOUT	OTG_FS_VBUS
43	69	102	D15	121	PA10	I/O	FT		USART1_RX/ TIM1_CH3/ OTG_FS_ID/DCMI_D1/ EVENTOUT	

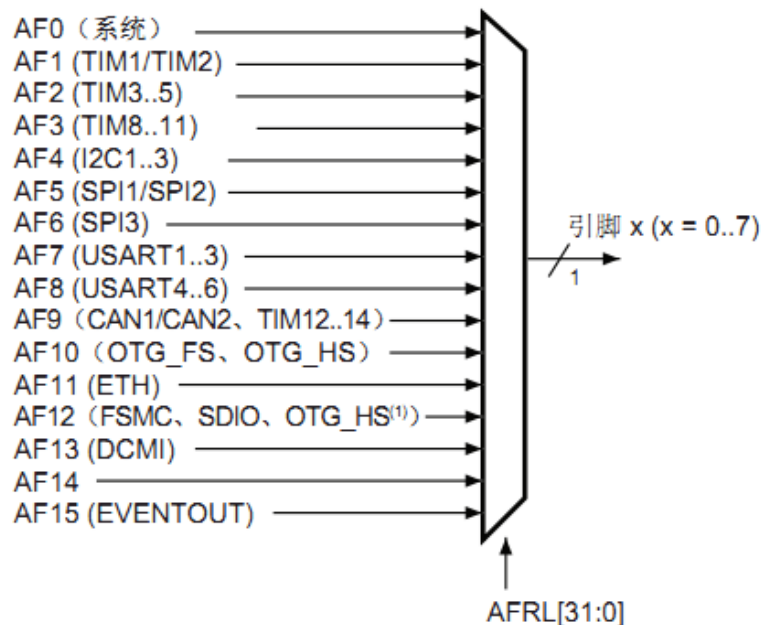
✓ 1. 端口复用

◆ STM32F4的端口复用映射原理

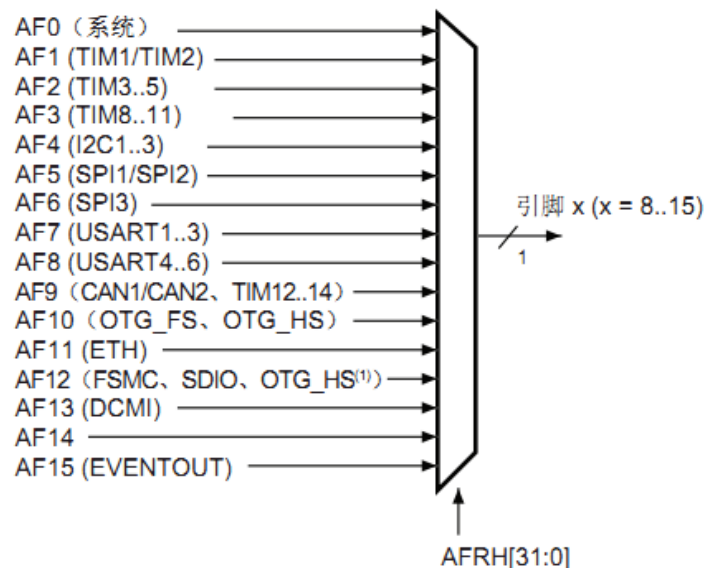
- ✓ STM32F4系列微控制器IO引脚通过一个复用器连接到内置外设或模块。该复用器一次只允许一个外设的复用功能（AF）连接到对应的IO口。这样可以确保共用同一个IO引脚的外设之间不会发生冲突。
- ✓ 每个IO引脚都有一个复用器，该复用器采用16路复用功能输入（AF0到AF15），可通过GPIOx_AFRL(针对引脚0-7)和GPIOx_AFRH（针对引脚8-15）寄存器对这些输入进行配置，每四位控制一路复用。

✓ 1. 端口复用

■ 端口复用映射示意图



对于引脚 8 到引脚 15, GPIOx_AFRH[31:0] 寄存器会选择专用的复用功能



✓ 1. 端口复用

■ AFRL 寄存器

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **AFRLy**: 端口 x 位 y 的复用功能选择 (Alternate function selection for port x bit y) (y = 0..7)
这些位通过软件写入，用于配置复用功能 I/O。

AFRLy 选择:

0000: AF0

0001: AF1

0010: AF2

0011: AF3

0100: AF4

0101: AF5

0110: AF6

0111: AF7

1000: AF8

1001: AF9

1010: AF10

1011: AF11

1100: AF12

1101: AF13

1110: AF14

1111: AF15

✓ 1. 端口复用

■ 复用功能映射配置

1. 系统功能

将 I/O 连接到 AF0，然后根据所用功能进行配置：

- JTAG/SWD：在各器件复位后，会将这些引脚指定为专用引脚，可供片上调试模块立即使用（不受 GPIO 控制器控制）。
- RTC_REFIN：此引脚应配置为输入浮空模式。
- MCO1 和 MCO2：这些引脚必须配置为复用功能模式。

2. GPIO

在 GPIOx_MODER 寄存器中将所需 I/O 配置为输出或输入。

✓ 1. 端口复用

3. 外设复用功能

对于 ADC 和 DAC，在 GPIOx_MODER 寄存器中将所需 I/O 配置为模拟通道。

对于其它外设：

- 在 GPIOx_MODER 寄存器中将所需 I/O 配置为复用功能
- 通过 GPIOx_OTYPER、GPIOx_PUPDR 和 GPIOx_OSPEEDER 寄存器，分别选择类型、上拉/下拉以及输出速度
- 在 GPIOx_AFRL 或 GPIOx_AFRH 寄存器中，将 I/O 连接到所需 AFx

✓ 1. 端口复用配置过程

◆ 端口复用为复用功能配置过程

-以PA9,PA10配置为串口1为例

①GPIO端口时钟使能。

RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA,ENABLE);

②复用外设时钟使能。

比如你要将端口PA9,PA10复用为串口，所以要使能串口时钟。

RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1,ENABLE);

③端口模式配置为复用功能。 **GPIO_Init（）** 函数。

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;//复用功能

✓ 1. 端口复用配置过程

④配置GPIOx_AFRL或者GPIOx_AFRH寄存器，将IO连接到所需的AFx。

```
/*PA9连接AF7，复用为USART1_TX */  
GPIO_PinAFConfig(GPIOA,GPIO_PinSource9,GPIO_AF_USART1);  
/* PA10连接AF7,复用为USART1_RX*/  
GPIO_PinAFConfig(GPIOA,GPIO_PinSource10,GPIO_AF_USART1);
```

✓ 1. 端口复用配置过程

◆ PA9,PA10复用为串口1的配置过程

```
RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA,ENABLE); //使能GPIOA时钟 ①  
RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1,ENABLE); //使能USART1时钟 ②
```

//USART1端口配置③

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9 | GPIO_Pin_10; //GPIOA9与GPIOA10  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF; //复用功能  
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //速度50MHz  
GPIO_InitStructure.GPIO_OType = GPIO_OType_PP; //推挽复用输出  
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP; //上拉  
GPIO_Init(GPIOA,&GPIO_InitStructure); //初始化PA9, PA10
```

//串口1对应引脚复用映射 ④

```
GPIO_PinAFConfig(GPIOA,GPIO_PinSource9,GPIO_AF_USART1); //GPIOA9复用为USART1  
GPIO_PinAFConfig(GPIOA,GPIO_PinSource10,GPIO_AF_USART1); //GPIOA10复用为USART1
```